

Algorithm for proof skeleton of equivalence step getting the goal for a single query after the call tactic has been applied using the translated Domino invariants:

Set up an empty proof

(1) while the goal has program code with branching instructions (if or match in our case):

(1.1) while next branching instruction in program on side {1} is an if statement:

```

if c1 = rndondt {1} ^if; 1: auto => /#. successful (does not give error back):
  if c2 = rndondt {2} ^if; 1: auto => /#. successful:
    add c1 and c2 to the proof and go to next while step of while loop (1)
  else:
    add c1 to the proof and go to next while step of while loop (1)

if c1 = rndondf {1} ^if; 1: auto => /#. successful (does not give error back):
  if c2 = rndondf {2} ^if; 1: auto => /#. successful:
    add c1 and c2 to the proof and go to next while step of while loop (1)
  else:
    add c1 to the proof and go to next while step of while loop (1)

if there exists a random sampling of variable vn1 before the next if in line l1 of program on side {1}:
  if there exists a random sampling of variable vn2 before the next if in line l2 of program on side {2}:
    if c = sp (l1-1) (l2-1); seq 1 1 : (#pre ^ vn1{1} = vn2{2}); 1: auto => /#. successful:
      add c to proof
    else ??? Check for which side might have 2, or how to relate samplings correctly ???
  add c = sp (l1-1) 0; seq 1 0: (#pre); 1: auto => /#. to proof

if program on side {2} has an if condition next
  try sp ^if ^if; if => //. and check for the subgoals:
    set n to be an empty set
    if first goal has only an expression and no program:
      if auto => /#. is successful:
        add 1 to the set n
        for all k greater 1 and smaller or equal the number of subgoals:
          if auto => /#. solves the subgoal successfully:
            add k to the set n
        add sp ^if ^if; if => //; n: auto => /#. to proof (here n is the set of solvable subgoals
        and needs to be added using commas like 1,3)
        if number of subgoals - |n| = 1:
          go to next step of while loop (1)
        else (in this case we branch into multiple subgoals):
          start algorithm for the unsolved subgoal(s) and use indentation
      else:
        (** the two if statements are not relatable and I do need to do something else...
        Maybe using human to solve this? **)
    for all k greater 0 and smaller or equal the number of subgoals:
      if auto => /#. solves the subgoal successfully:
        add k to the set n
    if set n is not empty:
      add sp ^if ^if; if => //; n: auto => /#. to proof and go to next while step
      if number of subgoals - |n| = 1:
        go to next step of while loop (1)
      else (in this case we branch into multiple subgoals):
        start algorithm for the unsolved subgoal(s) and use indentation
  else:
    add sp ^if 0; if {1} => //. to proof and start algorithm for subgoals. Use indentation!

```

(1.2) while next branching instruction in program on side {1} is a match statement:

```

for k greater 0 and smaller or equal the number of match patterns:
  if c1 = match mc_k {1} ^match => //; 1: auto => /#. successful (where mc_1 is the first pattern we match the variable
  with and so on)
  if c2 = match mc_k {2} ^match => //; 1: auto => /#. successful:
    add c1 and c2 to the proof and go to next while step of while loop (1)
  else:
    add c1 to the proof and go to next while step of while loop (1)

if there exists a random sampling of variable vn1 before the next match in line l1 of program on side {1}:
  if there exists a random sampling of variable vn2 before the next match in line l2 of program on side {2}:
    if c = sp (l1-1) (l2-1); seq 1 1 : (#pre ^ vn1{1} = vn2{2}); 1: auto => /#. successful:
      add c to proof
    else ??? Check for which side might have 2, or how to relate samplings correctly ???
  add c = sp (l1-1) 0; seq 1 0: (#pre); 1: auto => /#. to proof

if program on side {2} has an match next
  try sp ^if ^if; match => //. and check the subgoals:
    set n to be an empty set
    if first goal has only an expression and no program:
      if auto => /#. is successful:
        add 1 to the set n
        for all k greater 1 and smaller or equal the number of subgoals:

```

```

        if auto => /#. solves the subgoal successfully:
            add k to the set n
        add sp ^if ^if; match = => //; n: auto => /#. to proof
        if number of subgoals - |n| = 1:
            go to next step of while loop (1)
        else (in this case we branch into multiple subgoals):
            start algorithm for the unsolved subgoal(s) and use indentation
    else:
        (** the two if statements are not relatable and I do need to do something else...
        Maybe using human to solve this? **)
        for all k greater 0 and smaller or equal the number of subgoals:
            if auto => /#. solves the subgoal successfully:
                add k to the set n
        if set n is not empty:
            add sp ^if ^if; match = => //; n: auto => /#. to proof and go to next while step
            if number of subgoals - |n| = 1:
                go to next while step
            else (in this case we branch into multiple subgoals):
                start algorithm for the unsolved subgoal(s) and use indentation
    else:
        add sp ^if 0; match {1} => // to proof and start algorithm for subgoals. Use indentation!

```

(*** at this point there should not be any branchings left on side {1}, check for any left branches on side {2} ***)

(1.3) while next branching instruction in program on side {2} is an if statement:

```

        if c2 = rcondt {2} ^if; 1: auto => /#. successful:
            add c2 to the proof and go to next while step of while loop (1) which should lead to (1.3) again

        if c2 = rcondf {2} ^if; 1: auto => /#. successful:
            add c2 to the proof and go to next while step of while loop (1) which should lead to (1.3) again

        if there exists a random sampling of variable vn2 before the next if in line l2 of program on side {2}:
            add c = sp 0 (l2-1); seq 0 1: (#pre); 1: auto => /#. to proof

        add sp ^if 0; if {1} => // to proof and start algorithm for subgoals. Use indentation!

```

(1.4) while next branching instruction in program on side {2} is an match statement:

```

        for k greater 0 and smaller or equal the number of match patterns:
            if c2 = match mc_k {2} ^match => //; 1: auto => /#. successful:
                add c2 to the proof and go to next while step of while loop (1) which should lead to (1.3) again

        if there exists a random sampling of variable vn2 before the next match in line l2 of program on side {2}:
            add c = sp 0 (l2-1); seq 0 1: (#pre); 1: auto => /#. to proof

        add sp ^if 0; match {1} => // to proof and start algorithm for subgoals. Use indentation!

```

```

add auto => /> &1 &2 *. to the proof.
try do split and check subgoals:
    set n to be an empty set
    for all k greater 0 and smaller or equal the number of subgoals:
        if smt(). solves the subgoal successfully (could also be tried with specific lemmas in the smt call):
            add k to the set n
    if set n is not empty:
        add do split; n: smt(). to the proof
    else:
        add do split. to the proof

```